



**REENIDHI**  
EDUCATIONAL GROUP

**REENIDHI**  
INSTITUTE OF  
SCIENCE AND  
TECHNOLOGY



# COURSE ARTIFICIAL INTELLIGENCE AND DEEP LEARNING (AIDL)

## UNIT-V

Mr. P Ramu, Assistant Professor

# UNIT – V

## **BASICS OF DEEP LEARNING**

**1. Deep learning architectures:  
Convolutional Neural Networks :  
Neurons in Human Vision**

**2. The Shortcomings of Feature  
Selection**

**3.Full Description of the  
Convolutional Layer**

**4. Max Pooling**

**5. Full Architectural Description of  
Convolution Networks**

**6. Closing the Loop on MNIST with  
Convolutional Networks**

**7.Building a Convolutional Network  
for CIFAR-10**

**8.Visualizing Learning in  
Convolutional Networks**

**9. Leveraging Convolutional Filters  
to Replicate Artistic Styles**

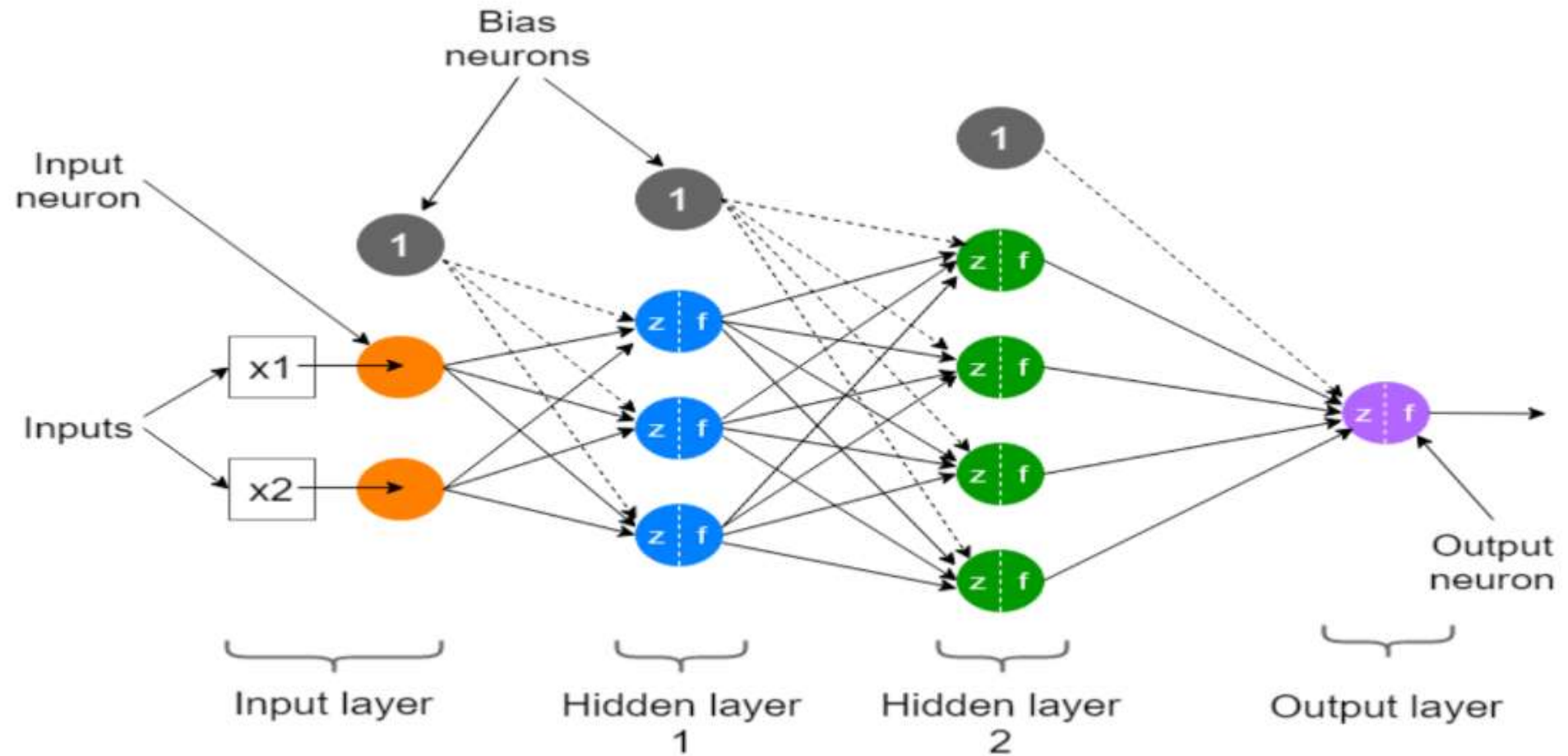
**10. Learning Convolutional Filters  
for Other Problem Domains**

**11. Training algorithms.**

# Introduction to Deep Learning

- Deep Learning is a subfield of Machine Learning and a part of Artificial Intelligence that uses **artificial neural networks with many layers** to automatically learn patterns from large amounts of data.
- It is inspired by the **structure and functioning of the human brain**, where interconnected neurons process information and learn from experience.

# Deep learning architectures



## Basic Idea of Deep Learning

Deep learning models learn **hierarchical representations of data**.

This means the model learns features step-by-step through multiple layers.

Example in image recognition:

1. **First layer** – detects edges
2. **Second layer** – detects shapes
3. **Third layer** – detects objects
4. **Final layer** – classifies the image

# Key Components of Deep Learning

## 1. Artificial Neural Networks (ANN)

The main structure used in deep learning consisting of interconnected neurons.

## 2. Layers

- Input Layer
- Hidden Layers
- Output Layer

## 3. Activation Functions

Functions like ReLU, Sigmoid, and Tanh introduce non-linearity.

## 4. Loss Function

Measures the error between predicted output and actual output.

## 5. Backpropagation Algorithm

Used to update weights and train the neural network.

## Why Deep Learning is Important

Deep learning can **automatically extract features from raw data**, unlike traditional machine learning which requires manual feature selection.

Advantages include:

- Handles **large and complex datasets**
- Produces **high accuracy results**
- Works well for **images, speech, and text**

## Applications of Deep Learning

Deep learning is used in many real-world applications:

- Image Recognition
- Speech Recognition
- Natural Language Processing
- Self-Driving Cars
- Medical Diagnosis
- Recommendation Systems

Example datasets used in deep learning include

MNIST dataset and

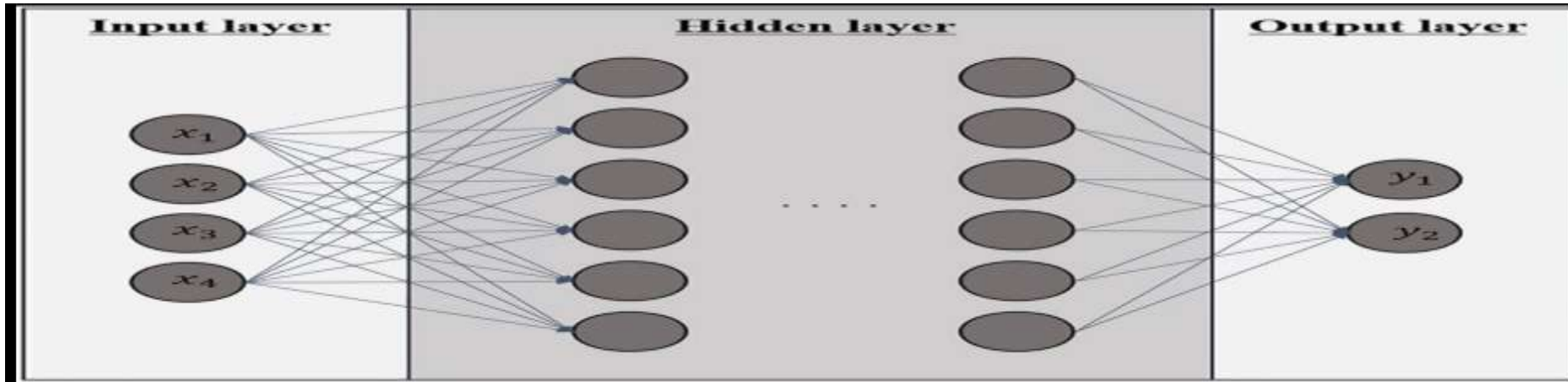
CIFAR-10 dataset.



# **Types of Deep Learning**

- 1. Artificial Neural Networks (ANN)**
- 2. Convolutional Neural Networks (CNN)**
- 3. Recurrent Neural Networks (RNN)**
- 4. Long Short-Term Memory (LSTM)**
- 5. Generative Adversarial Networks (GAN)**

# 1. Artificial Neural Networks (ANN)



Artificial Neural Networks (ANN) are the basic model of deep learning.

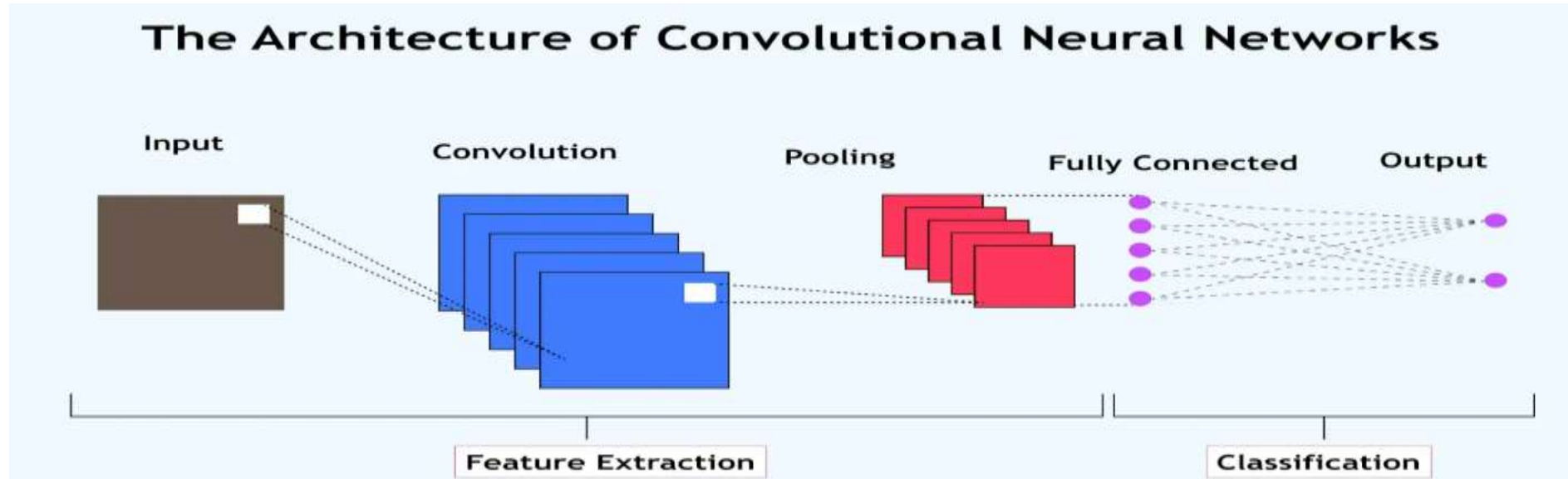
Features

- Consists of input, hidden, and output layers
- Each neuron is connected with weighted links
- Uses activation functions to produce output

Applications

- Pattern recognition
- Financial forecasting
- Medical diagnosis
- Classification problems

# 2. Convolutional Neural Networks (CNN)



**Convolutional Neural Networks (CNNs)** are mainly used for **image and visual data processing** in Deep Learning.

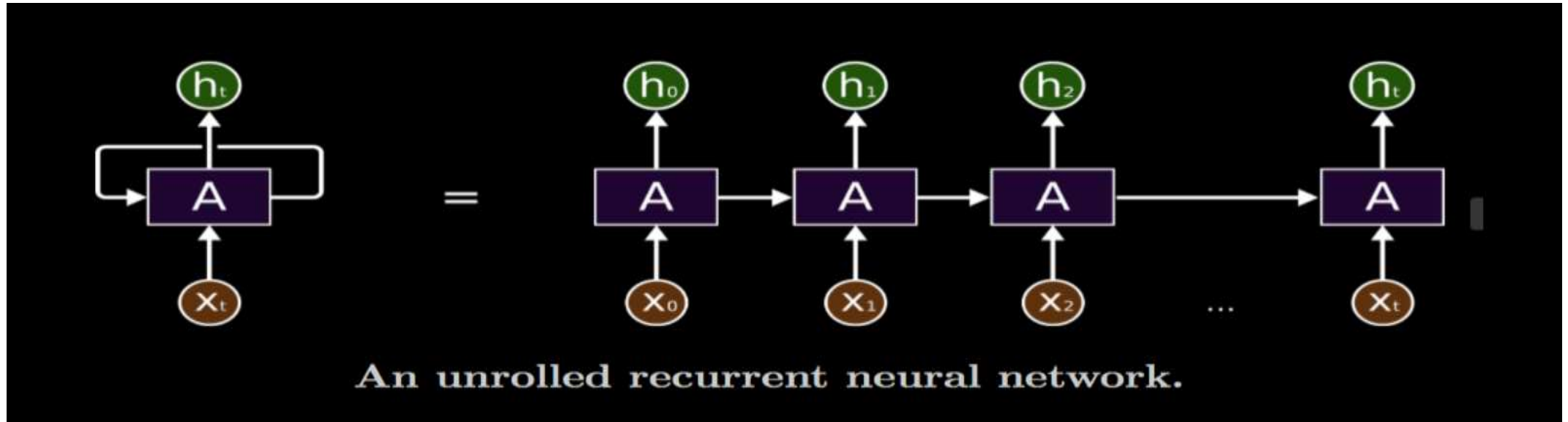
## Features

- Uses **convolution filters** to detect features
- Extracts edges, shapes, and textures
- Uses pooling layers to reduce feature map size

## Applications

- Image recognition
- Face detection
- Medical image analysis
- Self-driving cars

# 3. Recurrent Neural Networks (RNN)



**Recurrent Neural Networks (RNNs)** are designed to process **sequential data**.

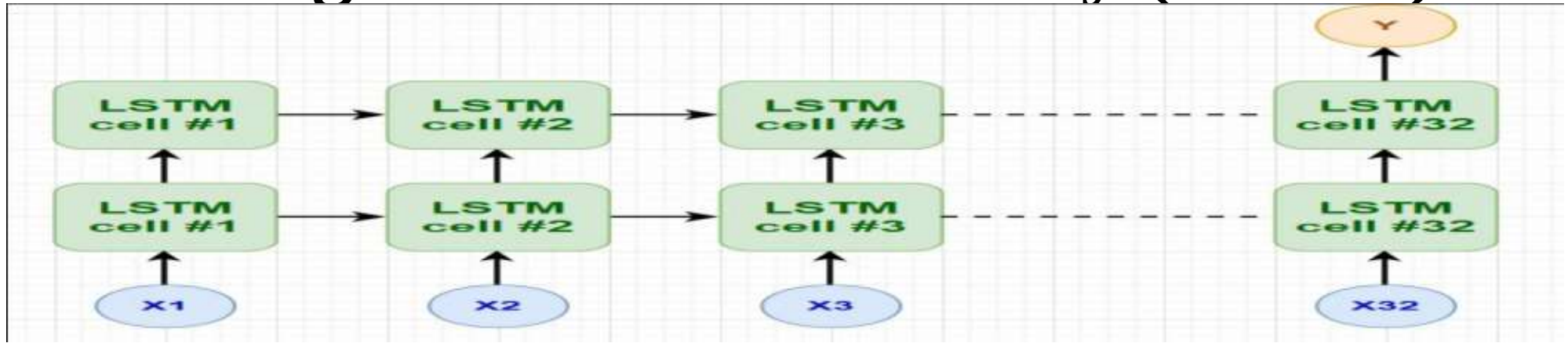
## Features

- Uses **feedback connections**
- Maintains memory of previous inputs
- Suitable for time-dependent data

## Applications

- Speech recognition
- Language translation
- Text generation
- Time-series prediction

# 4. Long Short-Term Memory (LSTM)



**Long Short-Term Memory (LSTM)** is an advanced type of RNN that solves the **long-term dependency problem**.

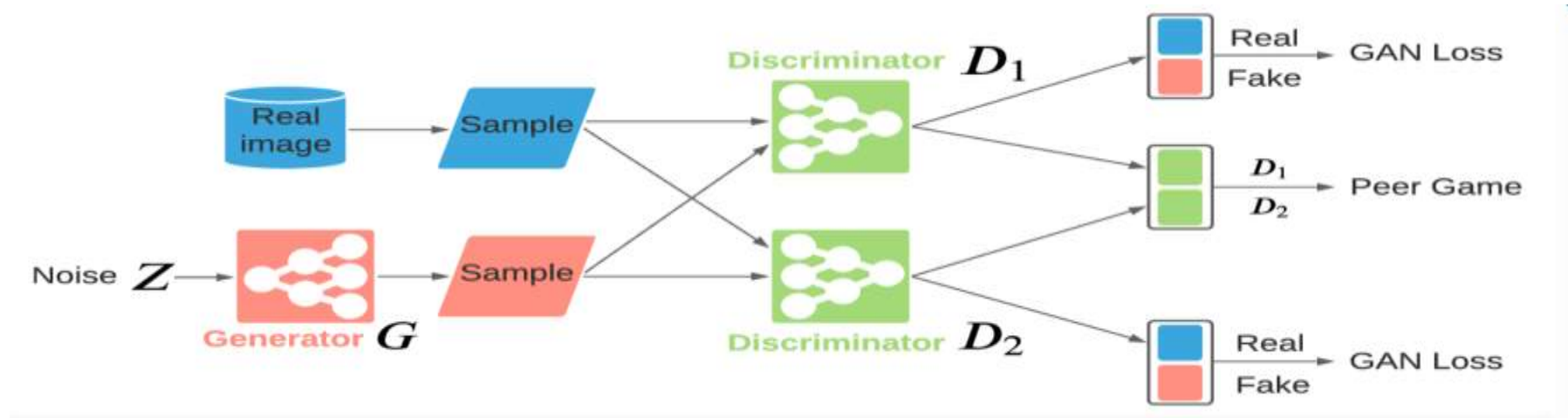
## Features

- Uses **memory cells**
- Contains three gates:
  - Input gate
  - Forget gate
  - Output gate

## Applications

- Machine translation
- Speech recognition
- Stock prediction
- Text prediction

# 5. Generative Adversarial Networks (GAN)



**Generative Adversarial Networks (GANs)** are used to **generate new data similar to training data**.

## Structure

- **Generator** – creates fake data
- **Discriminator** – distinguishes real data from fake data

## Applications

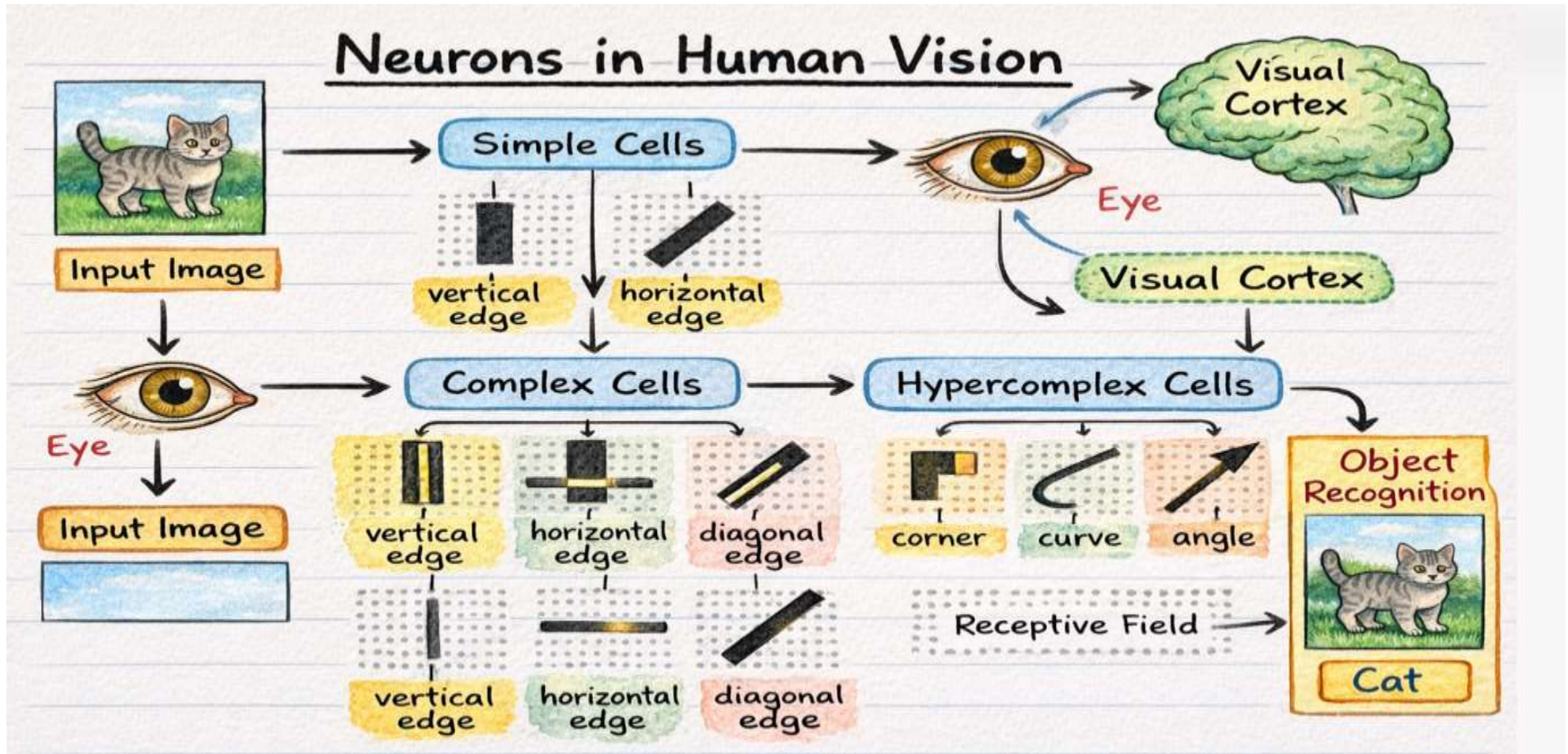
- Image generation
- Deepfake videos
- Image enhancement
- AI-generated art

# Summary Table

| Type | Purpose            | Example Applications |
|------|--------------------|----------------------|
| ANN  | General prediction | Pattern recognition  |
| CNN  | Image processing   | Face recognition     |
| RNN  | Sequential data    | Speech recognition   |
| LSTM | Long sequences     | Language translation |
| GAN  | Data generation    | AI image creation    |



# 1. Convolutional Neural Networks: Neurons in Human Vision





# Human Visual Processing System

In humans, vision is processed in the **visual cortex**:

## **Steps:**

1. Eyes capture light and form an image
2. Signals are sent to the brain
3. Neurons detect features like edges, colors, motion
4. Brain combines features → object recognition

# Types of Neurons in Human Vision

## 1. Simple Cells

- Detect **edges and orientations**
- Respond to specific patterns (horizontal, vertical, diagonal lines)
- Example: activates only for a **vertical edge**

## 2. Complex Cells

- Detect **patterns irrespective of exact position**
- Respond to **moving edges**
- Combine outputs from simple cells

## 3. Hypercomplex Cells

- Detect **complex shapes** like corners, curves
- Help in recognizing objects

# Receptive Field

- A **receptive field** is the **region of the visual field** a neuron responds to.
- Each neuron sees only a **small part of the image**
- Multiple neurons combine → full image understanding
- In CNNs, this is similar to **filters (kernels)** scanning small regions.

# Mapping Human Vision to CNN

Human Vision

CNN Equivalent

Neurons

Artificial neurons

Receptive field

Convolution filter

Edge detection

Feature maps

Hierarchical processing

Deep layers

## Hierarchical Feature Learning in CNN

- CNN mimics human vision by learning features in stages:

**First Layer** → edges, lines

**Second Layer** → textures

**Deeper Layers** → shapes, objects

**Final Layer** → classification

## Example: Human Vision

- To recognize a **cat**:
- Detect edges → ears
- Detect shapes → face
- Combine → identify as cat

## 2. The Shortcomings of Feature Selection

**Feature Selection** is the process of selecting the most relevant input variables (features) for building a model.

While it is useful, it has several **limitations**, especially compared to modern approaches in Deep Learning like **CNNs**, which automatically learn features.

# Major Shortcomings of Feature Selection

1. Manual Effort Required.
2. Loss of Important Information.
3. Not Scalable for Large Data.
4. Difficulty in Handling Complex Data.
5. Dependency on Data Representation.
6. Static Features (No Learning).
7. Limited Performance in Image Tasks.
8. Risk of Overfitting.

## 1. Manual Effort Required

- Requires domain knowledge and human expertise
- Time-consuming process
- Different datasets need different feature sets

👉 Example: In image processing, manually selecting edges, textures, etc., is difficult.

---

## 2. Loss of Important Information

- Removing features may discard useful hidden patterns
- Important relationships between features may be lost

👉 This can reduce model accuracy.

## 3. Not Scalable for Large Data

- Modern datasets (images, videos, text) have high dimensionality
  - Feature selection becomes computationally expensive
- 

## 4. Difficulty in Handling Complex Data

- Cannot effectively capture:
  - Non-linear relationships
  - Spatial patterns (in images)
  - Temporal patterns (in sequences)

👉 Traditional models fail in complex tasks like image recognition.



## 5. Dependency on Data Representation

- Performance depends on how features are represented
  - Poor feature design → poor model performance
- 

## 6. Static Features (No Learning)

- Features are fixed after selection
- Model cannot adapt or improve feature representation during training

👉 Unlike deep learning models that learn features automatically.

## 7. Limited Performance in Image Tasks

- In image data, manual features like:
  - edges
  - corners
  - texturesare **not sufficient** for accurate recognition

👉 This led to the development of CNNs.

---

## 8. Risk of Overfitting

- Selecting features based only on training data may cause:
  - Overfitting
  - Poor generalization to new data

# Why Deep Learning Overcomes These Issues

Modern models like CNNs in Deep Learning:

1. Automatically **learn hierarchical features**
2. Do not require manual feature selection
3. Handle complex data (images, videos) efficiently
4. Improve performance with large datasets

# **3. Full Description of the Convolutional Layer**

## **Concept of Full Description of the Convolutional Layer**

1. Introduction.
2. What is Convolution?
3. Components of Convolutional Layer.
4. Convolution Operation (Step-by-Step Example).
5. Output Size Formula.
6. Multiple Filters.
7. Activation Function.
8. Advantages.

## 1. Introduction

The **Convolutional Layer** is the **core building block** of a **Convolutional Neural Network (CNN)** in **Deep Learning**.

It is responsible for **extracting features** such as edges, textures, and patterns from input images.

---

## 2. What is Convolution?

Convolution is a mathematical operation where a **filter (kernel)** is applied to an input image to produce a **feature map**.

- The filter slides over the image
- Performs **element-wise multiplication**
- Adds the results to produce a single value

### 3. Components of Convolutional Layer

#### (a) Input Image

- Example:  $5 \times 5$  image matrix

#### (b) Filter (Kernel)

- Small matrix (e.g.,  $3 \times 3$ )
- Detects specific features like edges

Example kernel:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

👉 Detects vertical edges

### (c) Stride (S)

- Number of pixels the filter moves each step
  - Stride = 1 → moves one pixel at a time
  - Stride = 2 → skips pixels (faster, smaller output)
- 

### (d) Padding (P)

- Adding extra pixels (usually zeros) around the image

Types:

- **Valid Padding** → no padding
  - **Same Padding** → output size same as input
- 

### (e) Feature Map (Output)

- Result after applying filter across the image
- Shows presence of detected features

Source layer

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 5 | 2 | 6 | 8 | 2 | 0 | 1 | 2 |
| 4 | 3 | 4 | 5 | 1 | 9 | 6 | 3 |
| 3 | 9 | 2 | 4 | 7 | 7 | 6 | 9 |
| 1 | 3 | 4 | 6 | 8 | 2 | 2 | 1 |
| 8 | 4 | 6 | 2 | 3 | 1 | 8 | 8 |
| 5 | 8 | 9 | 0 | 1 | 0 | 2 | 3 |
| 9 | 2 | 6 | 6 | 3 | 6 | 2 | 1 |
| 9 | 8 | 8 | 2 | 6 | 3 | 4 | 5 |

Convolutional kernel

|    |    |   |
|----|----|---|
| -1 | 0  | 1 |
| 2  | 1  | 2 |
| 1  | -2 | 0 |

Destination layer

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |

$$\begin{aligned}
 &(-1 \times 5) + (0 \times 2) + (1 \times 6) + \\
 &(2 \times 4) + (1 \times 3) + (2 \times 4) + \\
 &(1 \times 3) + (-2 \times 9) + (0 \times 2) = 5
 \end{aligned}$$

Input

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 2 | 0 |
| 0 | 3 | 4 | 5 | 0 |
| 0 | 6 | 7 | 8 | 0 |
| 0 | 0 | 0 | 0 | 0 |

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

\*

=

Output

|    |    |    |    |
|----|----|----|----|
| 0  | 3  | 8  | 4  |
| 9  | 19 | 25 | 10 |
| 21 | 37 | 43 | 16 |
| 6  | 7  | 8  | 0  |

## 4. Convolution Operation (Step-by-Step Example)

Input (5×5) and Kernel (3×3):

1. Place kernel on top-left of image
  2. Multiply corresponding elements
  3. Sum all values
  4. Move kernel (based on stride)
  5. Repeat for entire image
- 

## 5. Output Size Formula

The size of output feature map is:

$$\frac{(N - F + 2P)}{S} + 1$$

Where:

- $N$  = Input size
- $F$  = Filter size
- $P$  = Padding
- $S$  = Stride



## 6. Multiple Filters

- CNN uses **multiple filters**
- Each filter detects different features:
  - edges
  - textures
  - colors

👉 Output = multiple feature maps

---

## 7. Activation Function

After convolution, an activation function like **ReLU** is applied:

- Removes negative values
- Introduces **non-linearity**

## 8. Advantages

- Automatic feature extraction
- Fewer parameters than fully connected layers
- Efficient for image processing

# 4. Max Pooling

## 1. Definition

**Max Pooling** is a **downsampling operation** used in CNNs that reduces the size of feature maps by selecting the **maximum value** from a small region.

It is commonly used in Deep Learning models for efficient image processing.

---

## 2. How Max Pooling Works

- A window (e.g.,  $2 \times 2$ ) slides over the feature map
- From each region, it selects the **maximum value**
- Produces a smaller output feature map



### 3. Example

Input Feature Map ( $4 \times 4$ )

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 3 & 2 & 4 & 1 \\ 1 & 2 & 3 & 5 \end{bmatrix}$$

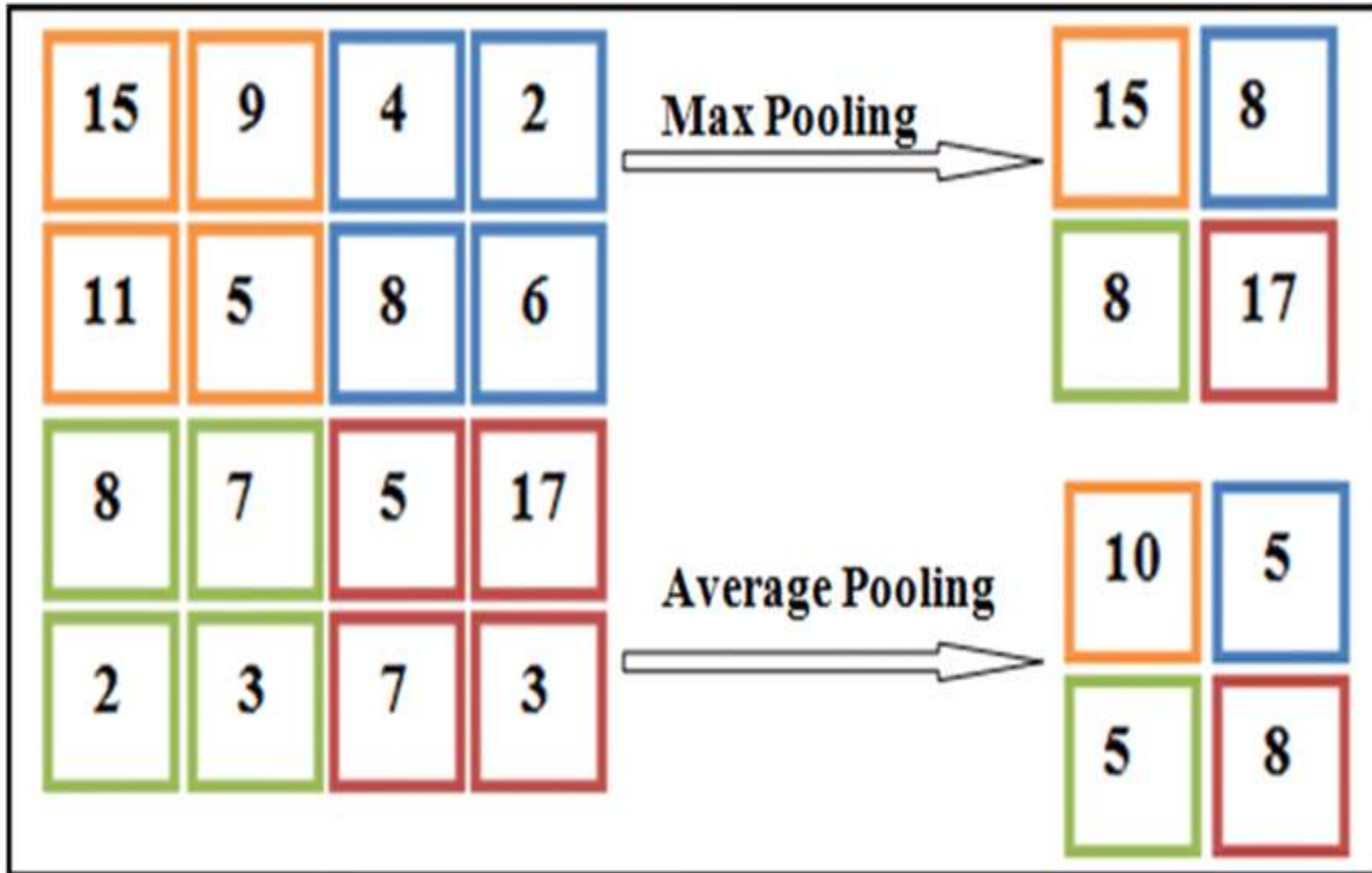
Apply  $2 \times 2$  Max Pooling (Stride = 2)

Step-by-step:

- Region 1  $\rightarrow \max(1, 3, 5, 6) = 6$
- Region 2  $\rightarrow \max(2, 4, 1, 2) = 4$
- Region 3  $\rightarrow \max(3, 2, 1, 2) = 3$
- Region 4  $\rightarrow \max(4, 1, 3, 5) = 5$

Output ( $2 \times 2$ )

$$\begin{bmatrix} 6 & 4 \\ 3 & 5 \end{bmatrix}$$



## 4. Parameters of Max Pooling

### 1. Pool Size

- Size of window (e.g.,  $2 \times 2$ ,  $3 \times 3$ )

### 2. Stride

- Step size of window movement
  - Usually same as pool size
- 

## 5. Advantages

- Reduces **computational complexity**
- Controls **overfitting**
- Keeps **important features** (max values)
- Makes model **translation invariant**

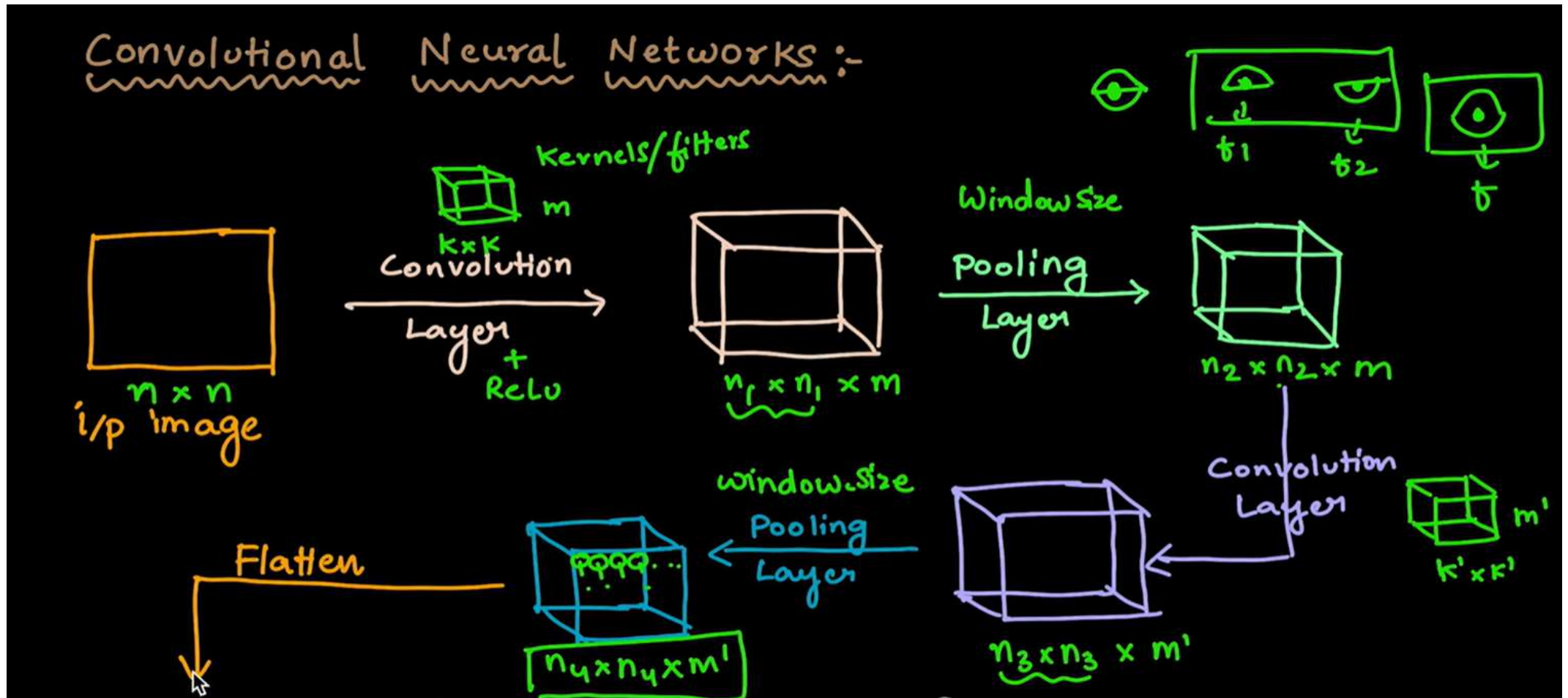
## 6. Disadvantages

- Loses some information
  - May ignore small but important details
- 

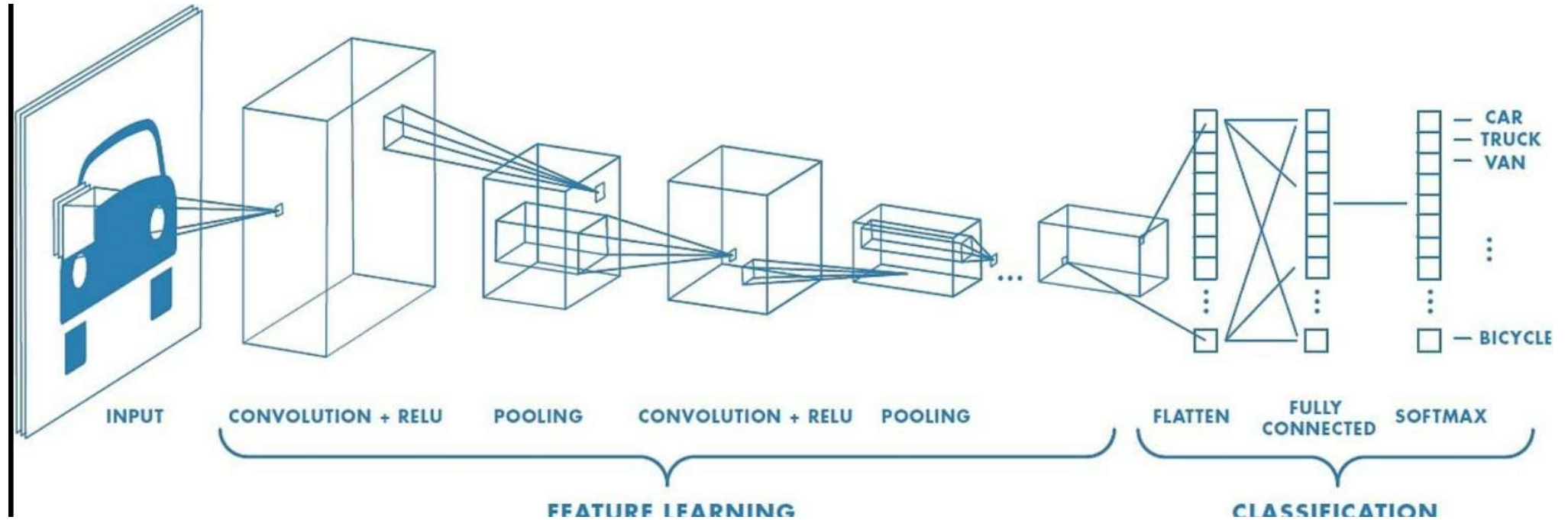
## 7. Types of Pooling (Comparison)

| Type            | Description         |
|-----------------|---------------------|
| Max Pooling     | Takes maximum value |
| Average Pooling | Takes average value |

# 5. Architecture of Convolutional Neural Networks



# Architecture of Convolutional Neural Networks





# Convolutional Neural Networks (CNN)

- A **Convolutional Neural Network (CNN)** is a deep learning model used mainly for **image recognition and computer vision tasks** in Deep Learning.
- CNN consists of several layers that work together to **extract features and perform classification**.

# Main Layers of CNN

- 1. Input Layer.**
- 2. Convolutional Layer.**
- 3. Activation Layer.**
- 4. Pooling Layer.**
- 5. Flatten Layer.**
- 6. Fully Connected Layer.**
- 7. Output Layer.**

# 1. Input Layer

The **input layer** takes data from the original data source, such as:

- **Images** (most common for CNN)
- **Video frames**
- **Medical images**
- **Satellite images**
- **Handwritten digit images** (e.g., MNIST dataset)

For example:

- In image classification, the input layer receives **pixel values of the image**.

## 2 Example of Image Input

If the image size is **28 × 28 pixels**, the input layer will take:

$$Input = 28 \times 28 \times 1$$

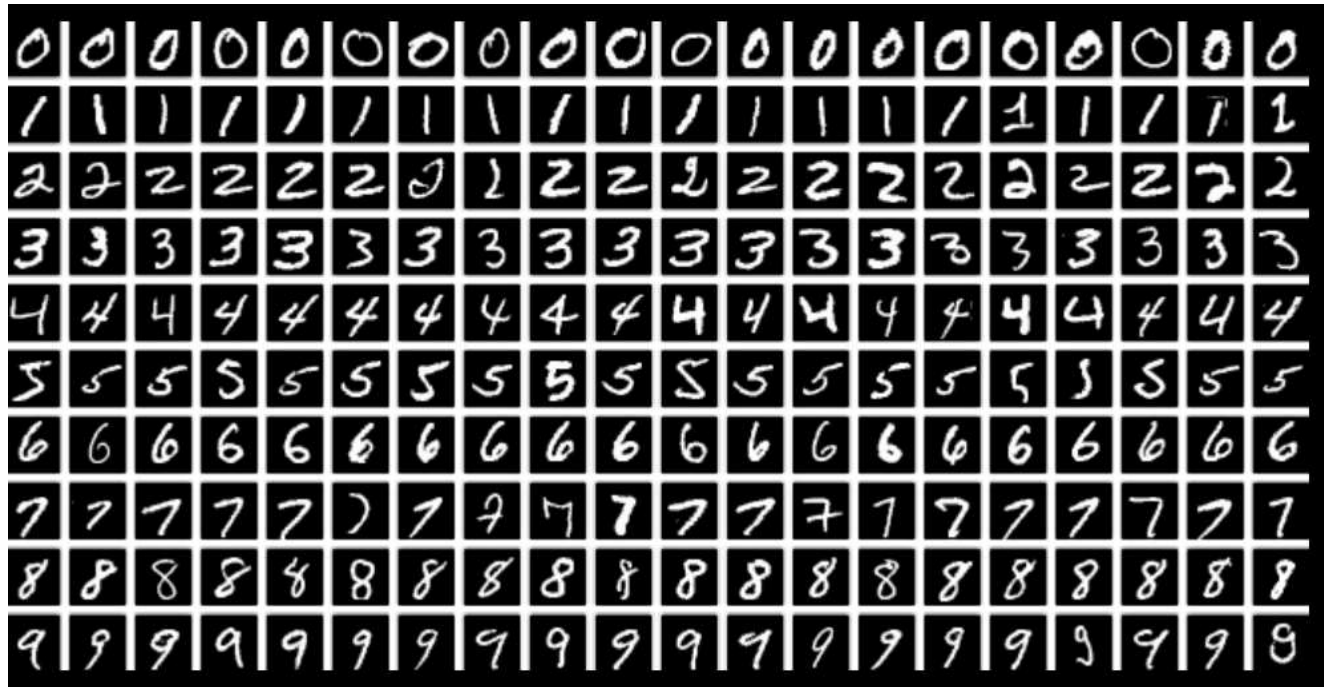
- **28** → height
- **28** → width
- **1** → grayscale channel

For a color image:

$$Input = 32 \times 32 \times 3$$

- **3 channels** → RGB (Red, Green, Blue).

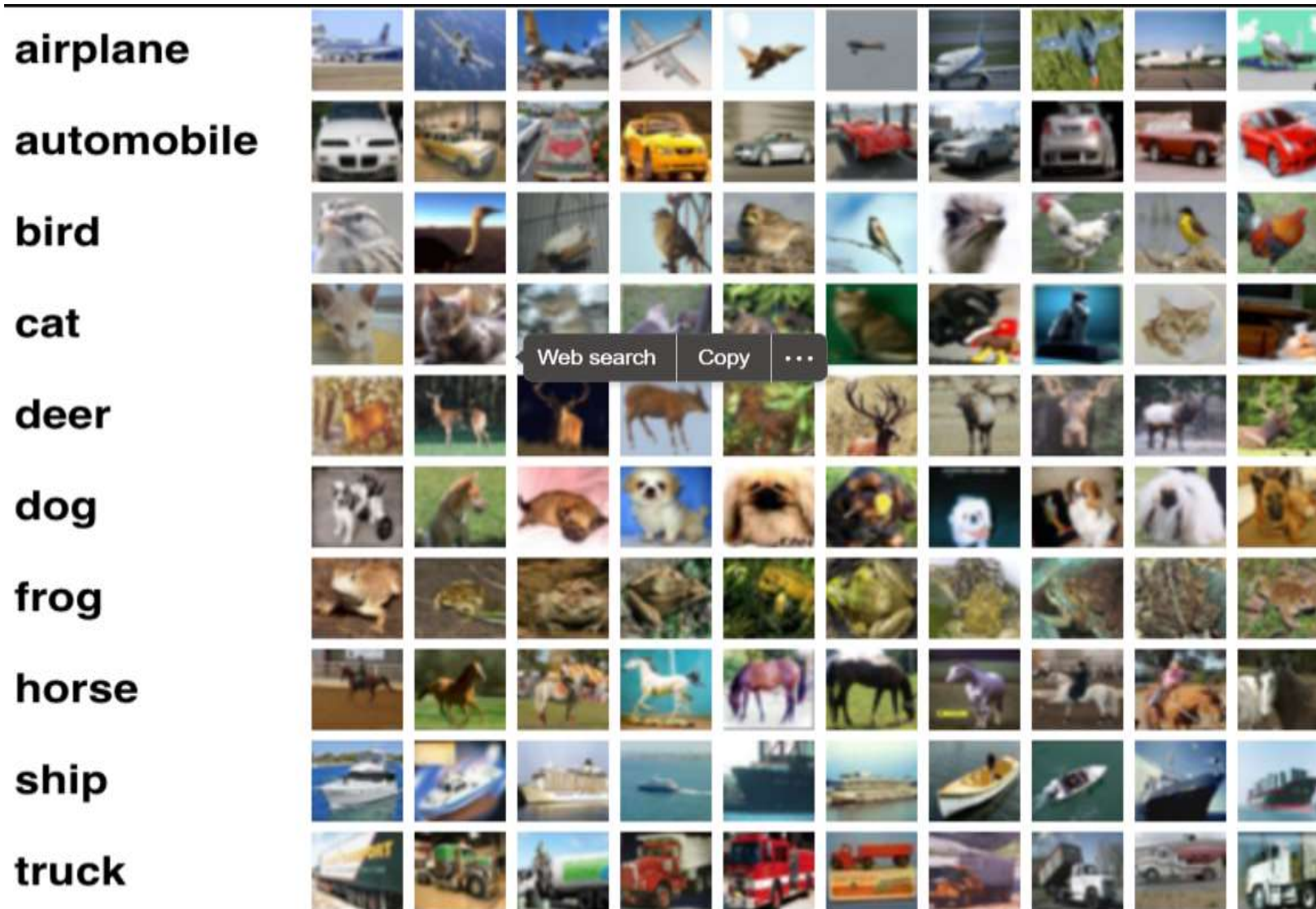
# MNIST dataset in CNN



## Characteristics:

- Contains **handwritten digit images (0–9)**
- Total images: **70,000**
- Training images: **60,000**
- Testing images: **10,000**
- Image size: **28 × 28 pixels**
- Grayscale images

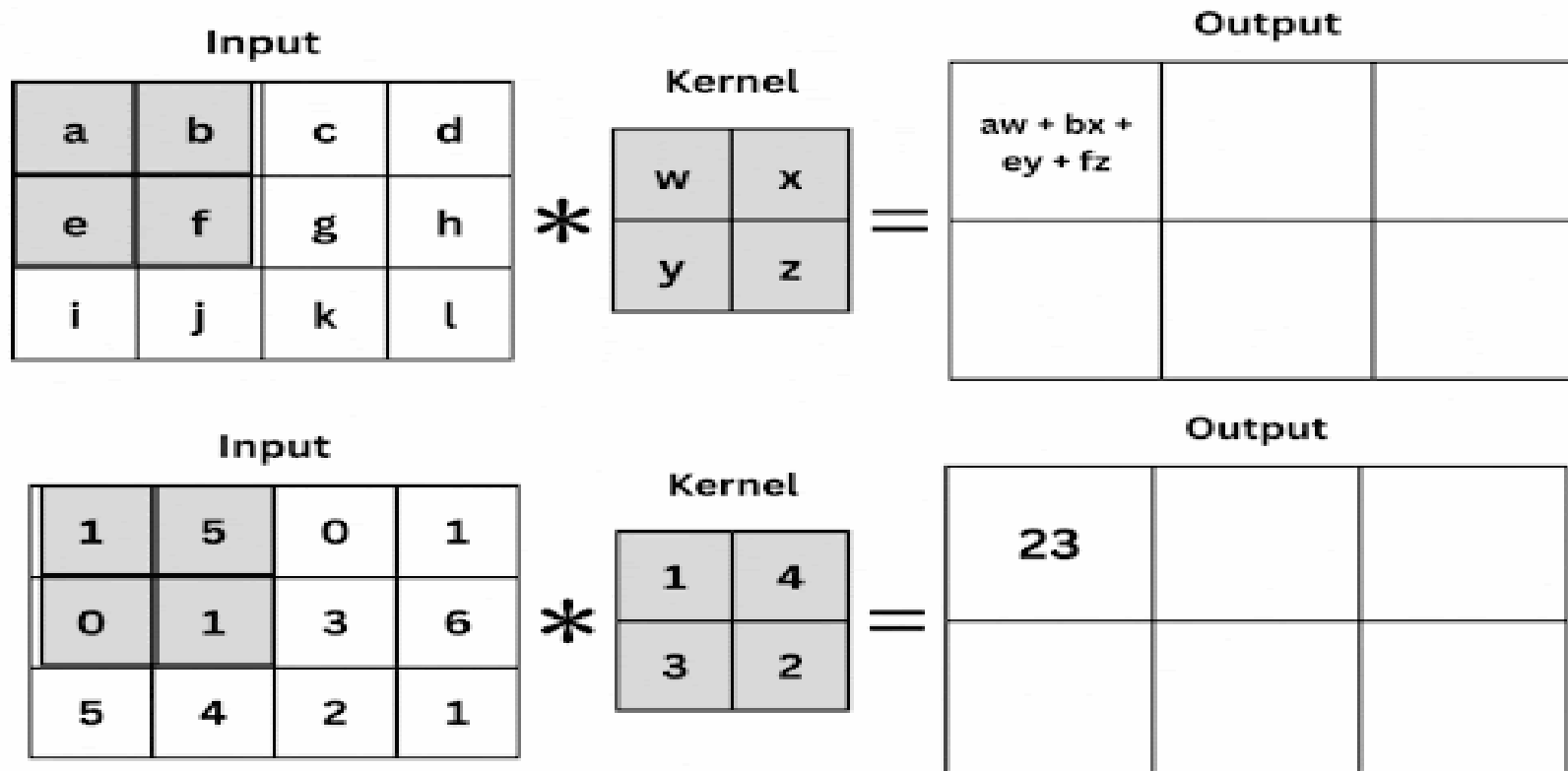
# CIFAR-10 dataset in CNN



## Characteristics:

- Contains **60,000 color images**
- Image size: **32 × 32 pixels**
- Training images: **50,000**
- Testing images: **10,000**
- 10 object categories

## 2. Convolutional Layer



## 2. Convolutional Layer

|    |   |    |    |
|----|---|----|----|
| 6  | 5 | -1 | -4 |
| 7  | 4 | 8  | 6  |
| 4  | 2 | -1 | 5  |
| -6 | 6 | 3  | 9  |

 $*$ 

|    |   |   |
|----|---|---|
| -1 | 5 | 2 |
| 5  | 3 | 1 |
| -1 | 0 | 3 |

 $=$ 

|    |  |
|----|--|
| 65 |  |
|    |  |

|    |    |    |    |
|----|----|----|----|
| 4  | 6  | 7  | -4 |
| -4 | -6 | 2  | 1  |
| 5  | 6  | -5 | 3  |
| 2  | 1  | 5  | 0  |

 $*$ 

|   |    |   |
|---|----|---|
| 4 | -2 | 2 |
| 6 | 1  | 3 |
| 0 | 0  | 2 |

 $=$ 

|     |  |
|-----|--|
| -16 |  |
|     |  |

|    |    |   |    |
|----|----|---|----|
| 3  | 6  | 2 | 1  |
| 2  | 4  | 6 | -9 |
| -5 | -5 | 3 | 4  |
| 5  | 2  | 1 | 8  |

 $*$ 

|   |   |   |
|---|---|---|
| 5 | 3 | 6 |
| 3 | 4 | 7 |
| 9 | 1 | 2 |

 $=$ 

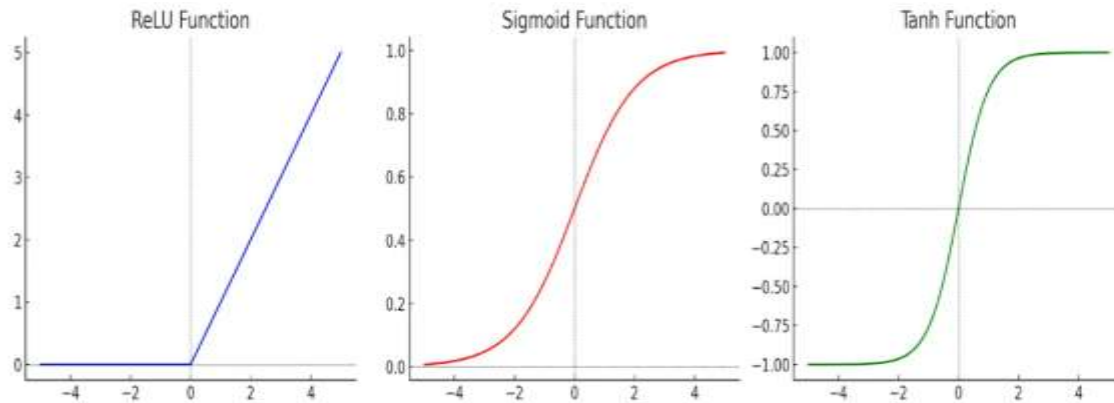
|    |  |
|----|--|
| 65 |  |
|    |  |



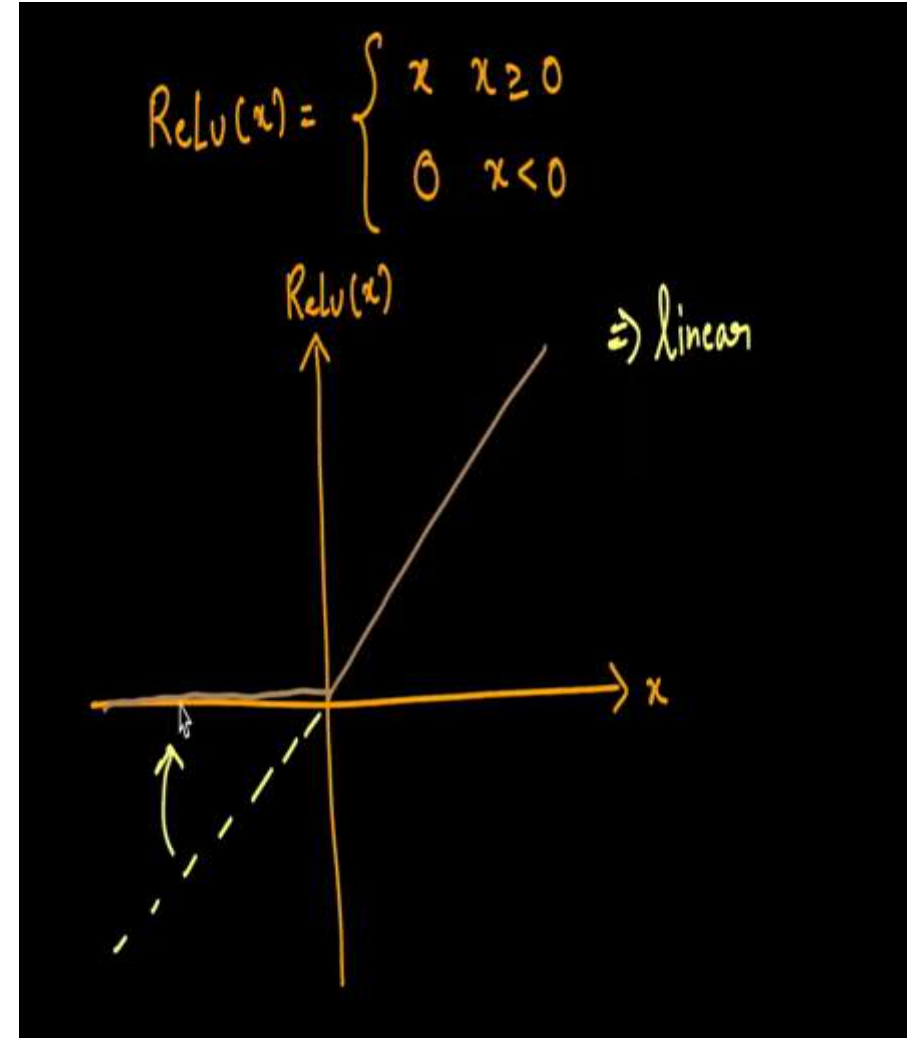
# 3. Activation Layer.

## Activation Function

An **Activation Function** introduces non-linearity into the network, allowing it to learn complex patterns beyond simple linear transformations. Common activation functions include **ReLU** (**Rectified Linear Unit**), **Sigmoid**, and **Tanh**, each with distinct properties suited for different tasks. Among them, **ReLU** is widely used due to its simplicity and efficiency. It converts all negative values in the feature map to zero while retaining positive values, helping the network train faster and mitigate the vanishing gradient problem.

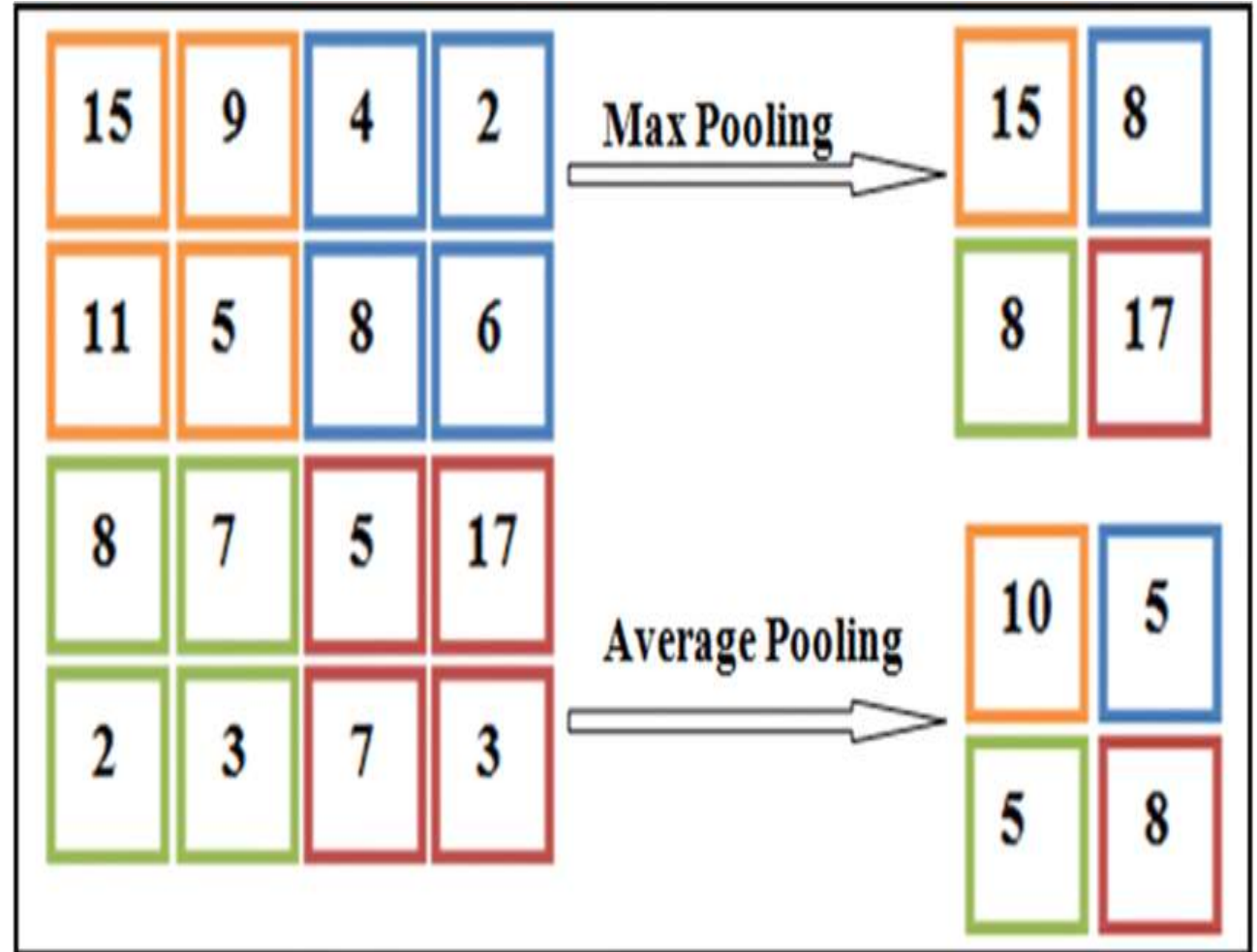
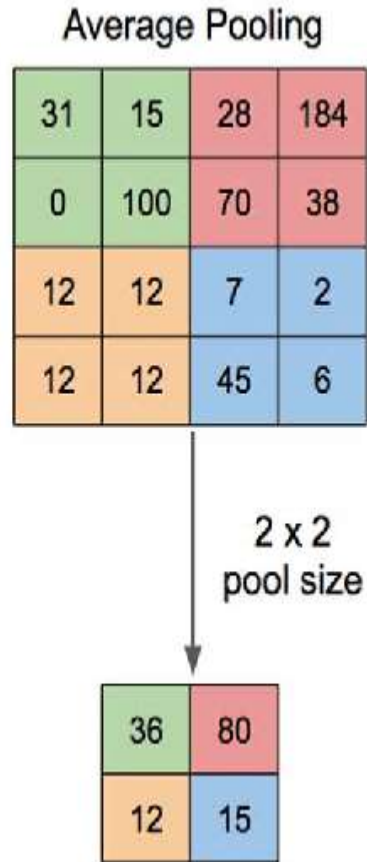
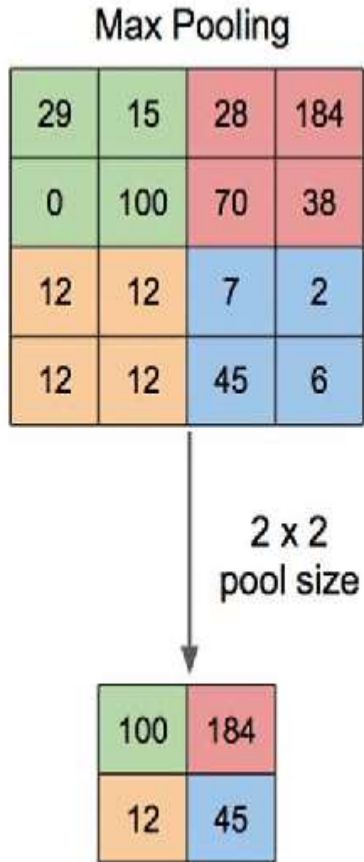


Title: Visualization of Activation Functions: ReLU, Sigmoid, and Tanh





# 4. Pooling Layer



# 5. Flatten Layer

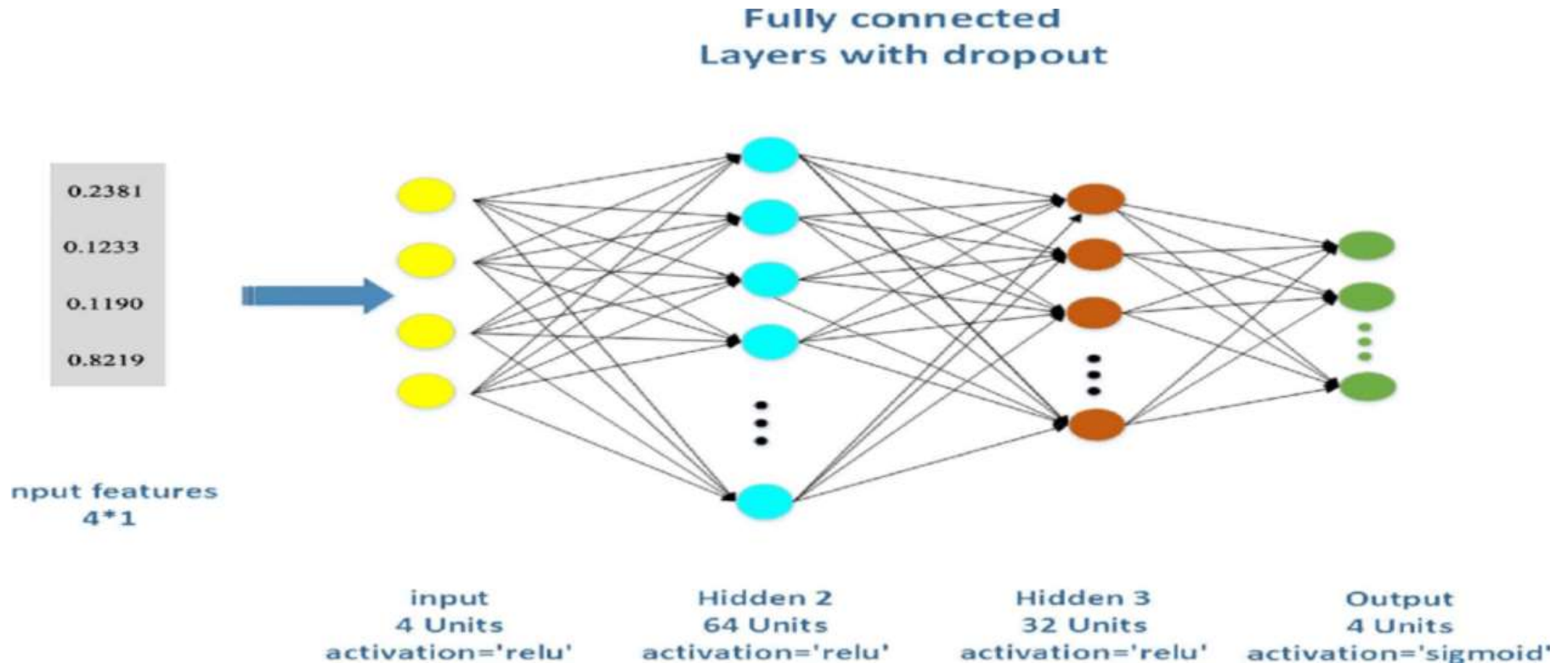
Flattening Layer is one of the simplest steps in CNN. To illustrate, let's consider an example where we have passed some input through each layer of the CNN and got a result like this,

$$MaxPool((I * K)(i, j)) = \begin{bmatrix} 1 & 3 & 5 \\ -4 & 0 & -3 \\ 5 & 7 & -9 \end{bmatrix}$$

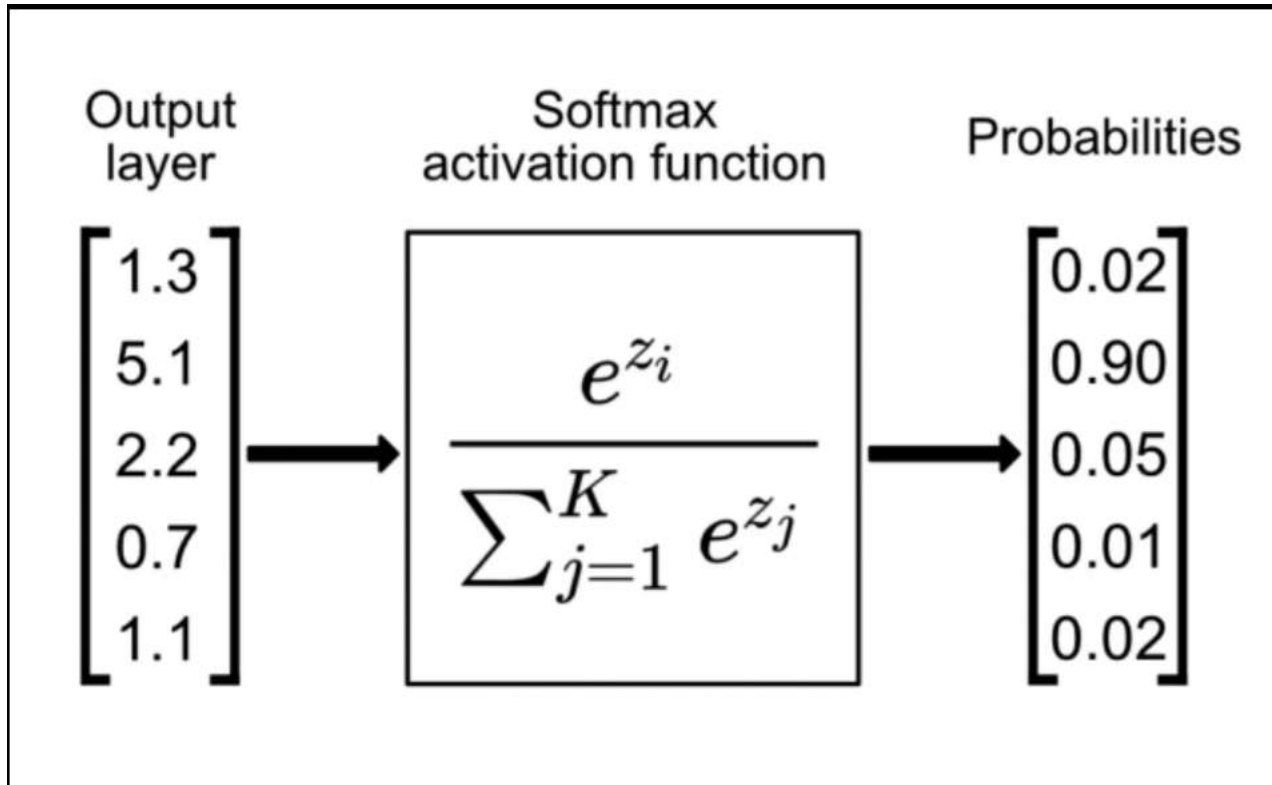
When this matrix is flattened it will look like this,

$$F = [1 \quad 3 \quad 5 \quad -4 \quad 0 \quad -3 \quad 5 \quad 7 \quad -9]$$

# 6. Fully Connected Layer



# Softmax in CNN



# 7. Output Layer

## 7. Output Layer (Softmax)

The **output layer** produces the final prediction using **Softmax activation**.

Example classes:

- Cat
- Dog
- Car

Softmax output:

| Class | Probability |
|-------|-------------|
| Cat   | 0.80        |
| Dog   | 0.15        |
| Car   | 0.05        |

Prediction → **Cat**

# 6. Closing the Loop on MNIST with Convolutional Networks

## 1. Introduction

**“Closing the Loop on MNIST with Convolutional Networks”** refers to using **Convolutional Neural Networks (CNNs)** to achieve **high accuracy in handwritten digit recognition** on the **MNIST** dataset.

- It demonstrates how CNNs **automatically learn features** and outperform traditional machine learning methods in Deep Learning.

## 2. About MNIST Dataset

- Contains 70,000 grayscale images
- Each image size: 28 × 28 pixels
- Classes: Digits (0–9)

Split:

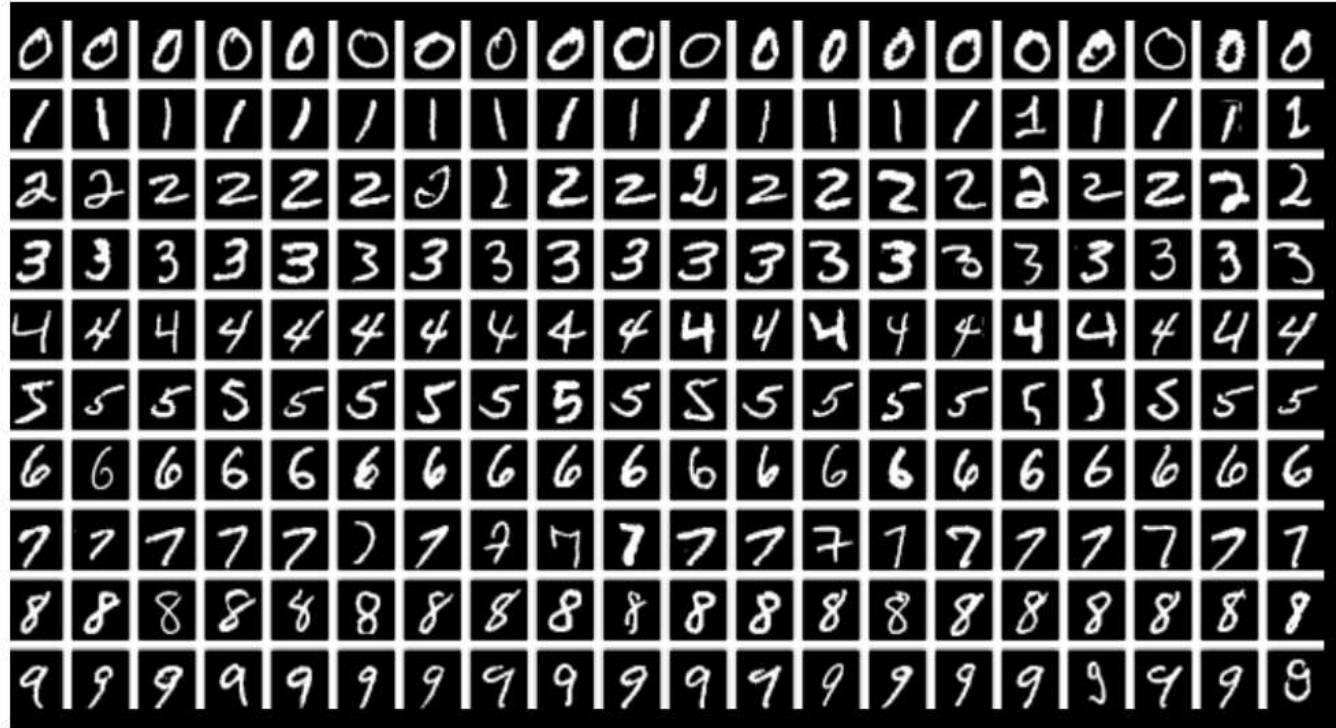
- 60,000 training images
  - 10,000 testing images
- 

## 3. Problem Statement

Given an input image of a handwritten digit, the goal is to:

👉 Correctly classify it into one of 10 classes (0–9)

# MNIST dataset in CNN



## Characteristics:

- Contains **handwritten digit images (0–9)**
- Total images: **70,000**
- Training images: **60,000**
- Testing images: **10,000**
- Image size: **28 × 28 pixels**
- Grayscale images

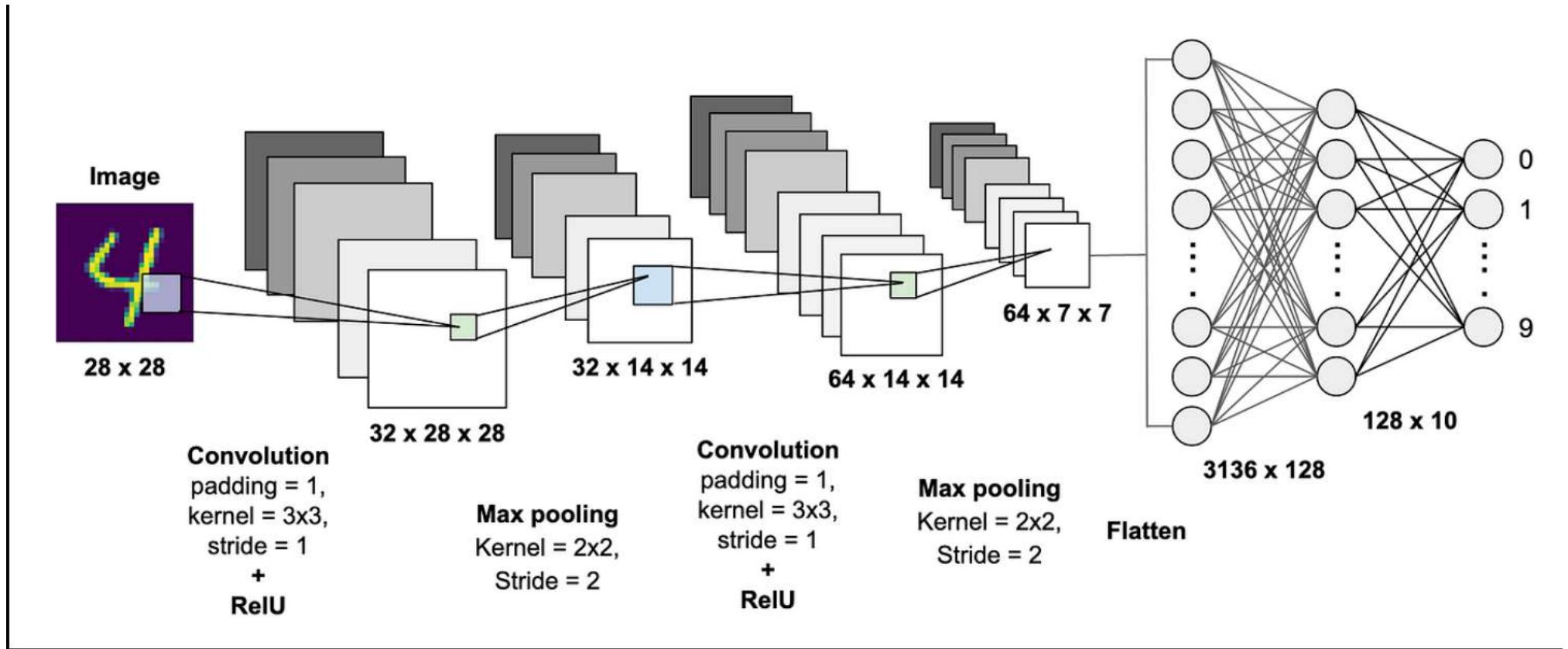


## 4. CNN Architecture for MNIST

Typical CNN model:

1. Input Layer → 28×28 image
2. Convolution Layer (3×3 filters)
3. ReLU Activation
4. Max Pooling (2×2)
5. Convolution Layer
6. ReLU Activation
7. Max Pooling
8. Flatten Layer
9. Fully Connected Layer
10. Output Layer (Softmax)

# 5. Working Process in MNIST



## 5. Working Process

### Step 1: Input

- A digit image (e.g., "5") is given as input

### Step 2: Feature Extraction

- Convolution layers detect:
  - edges
  - curves
  - strokes

### Step 3: Downsampling

- Pooling reduces image size while keeping important features

### Step 4: Classification

- Fully connected layer analyzes features
- Softmax outputs probabilities for digits (0–9)

## 6. Example

Input image: digit "3"

Output probabilities:

| Digit  | Probability  |
|--------|--------------|
| 0      | 0.01         |
| 1      | 0.02         |
| 2      | 0.05         |
| 3      | 0.90         |
| Others | Small values |

👉 Prediction = 3

## 7. Performance

- Traditional ML accuracy: ~90–95%
- CNN accuracy: ~99%+

👉 CNN "closes the loop" by achieving **near-human performance**

# 7. Building a Convolutional Network for CIFAR-10

## 1. Introduction

CIFAR-10 is a standard benchmark for image classification in Deep Learning.

It contains small **color images** ( $32 \times 32 \times 3$ ) across **10 classes** (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

---

## 2. Dataset Details

- 60,000 images
  - 50,000 training
  - 10,000 testing
- 10 classes (balanced)
- RGB images → more complex than grayscale datasets

**airplane**



**automobile**



**bird**



**cat**



**deer**



**dog**



**frog**



**horse**



**ship**



**truck**



### 3. Preprocessing Steps

Before building the CNN:

#### 1. Normalization

- Scale pixel values (0–255 → 0–1)

#### 2. Data Augmentation (important)

- Rotation
- Flipping
- Cropping

#### 3. Train-Test Split

- Use standard split or validation set
- 

### 4. CNN Architecture for CIFAR-10

A typical architecture:

#### Block 1

- Conv (32 filters,  $3 \times 3$ )
- ReLU
- Conv (32 filters,  $3 \times 3$ )
- ReLU
- Max Pooling ( $2 \times 2$ )

## Block 2

- Conv (64 filters,  $3 \times 3$ )
- ReLU
- Conv (64 filters,  $3 \times 3$ )
- ReLU
- Max Pooling

## Block 3

- Conv (128 filters,  $3 \times 3$ )
- ReLU
- Max Pooling

## Classification Head

- Flatten
- Fully Connected (Dense layer)
- Dropout (to prevent overfitting)
- Output Layer (Softmax – 10 classes)



## 5. Working Process

1. Input image ( $32 \times 32 \times 3$ )
2. Convolution → extract features (edges, colors)
3. ReLU → non-linearity
4. Pooling → reduce dimensions
5. Repeat convolution blocks → deeper features
6. Flatten → convert to vector
7. Fully connected layer → classification
8. Softmax → probability output

## 6. Loss Function and Optimizer

- **Loss Function:** Categorical Cross-Entropy
- **Optimizer:** Adam / SGD
- **Evaluation Metric:** Accuracy

## 7. Example Prediction

Input: Image of a dog

Output probabilities:

| Class  | Probability  |
|--------|--------------|
| Cat    | 0.10         |
| Dog    | 0.80         |
| Horse  | 0.05         |
| Others | Small values |

👉 Final Prediction: **Dog**

## 8. Challenges in CIFAR-10

✦ Get Plus ✕

- Small image size ( $32 \times 32$ )
  - Complex backgrounds
  - Similar classes (cat vs dog)
  - Color variations
- 

## 9. Techniques to Improve Accuracy

- Data augmentation
  - Batch normalization
  - Dropout
  - Deeper networks
  - Learning rate scheduling
- 

## 10. Performance

- Basic CNN: **70–80% accuracy**
  - Advanced CNN (ResNet, VGG): **90%+ accuracy**
- 

## 11. Applications

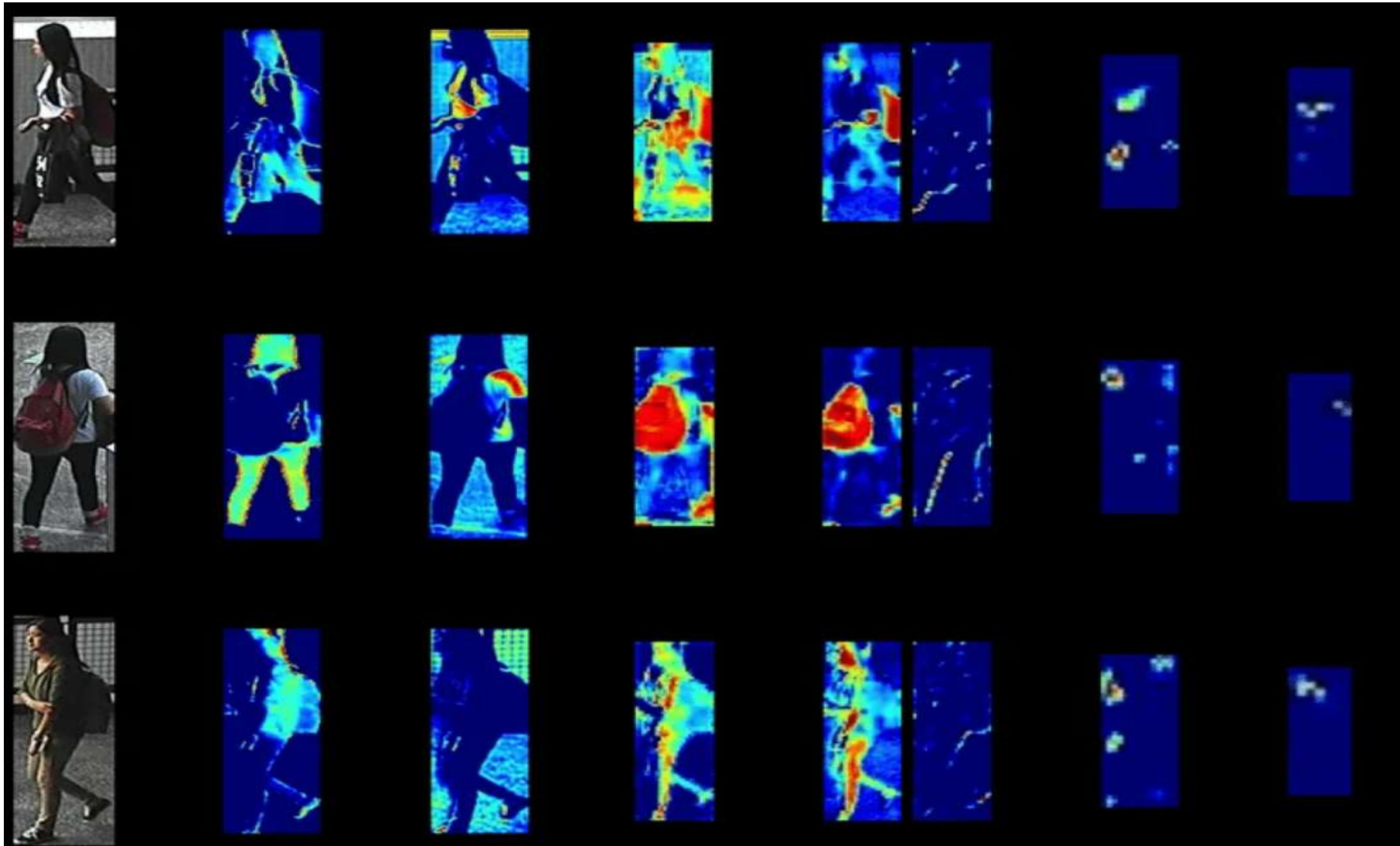
- Object recognition
- Autonomous systems
- Image tagging
- Surveillance systems

# 8. Visualizing Learning in Convolutional Networks

## 1. Introduction

**Visualizing learning in CNNs** means understanding **what a Convolutional Neural Network learns internally** while processing images.

Since CNNs are often called “**black-box models,**” visualization helps interpret their behavior in Deep Learning.



## 2. Why Visualization is Important

- Understand how CNN learns features
  - Debug model errors
  - Improve model performance
  - Increase interpretability and trust
- 

## 3. What Does a CNN Learn?

CNN learns features hierarchically:

### 1. Early Layers

- Edges
- Lines
- Corners

### 2. Middle Layers

- Textures
- Patterns

### 3. Deep Layers

- Object parts (eyes, wheels, etc.)
- Complete objects

## 4. Techniques for Visualization

### 4.1 Feature Map Visualization

- Shows output of each convolution layer
- Helps see what features are activated

👉 Example:

- First layer → edge maps
  - Deeper layers → complex patterns
- 

### 4.2 Filter (Kernel) Visualization

- Displays learned filters
- Helps understand what patterns filters detect

Example filters:

- Edge detectors
  - Color blobs
- 

### 4.3 Activation Maximization

- Finds input image that **maximally activates a neuron**
  - Shows what pattern the neuron is looking for
- 

### 4.4 Gradient-based Visualization (Saliency Maps)

- Highlights **important regions in input image**
- Shows where model is focusing

Example:

- In a cat image → highlights face/ears
- 

### 4.5 Grad-CAM (Class Activation Map)

- Produces heatmaps
- Shows **which parts influence prediction**

👉 Very popular for explainability



## 5. Example

Input: Image of a dog

Visualization results:

- Layer 1 → detects edges
  - Layer 2 → fur texture
  - Layer 3 → shape of face
  - Grad-CAM → highlights dog face
- 

## 6. Benefits

- Improves model transparency
- Helps detect overfitting
- Useful in sensitive fields like medical AI

## 7. Limitations

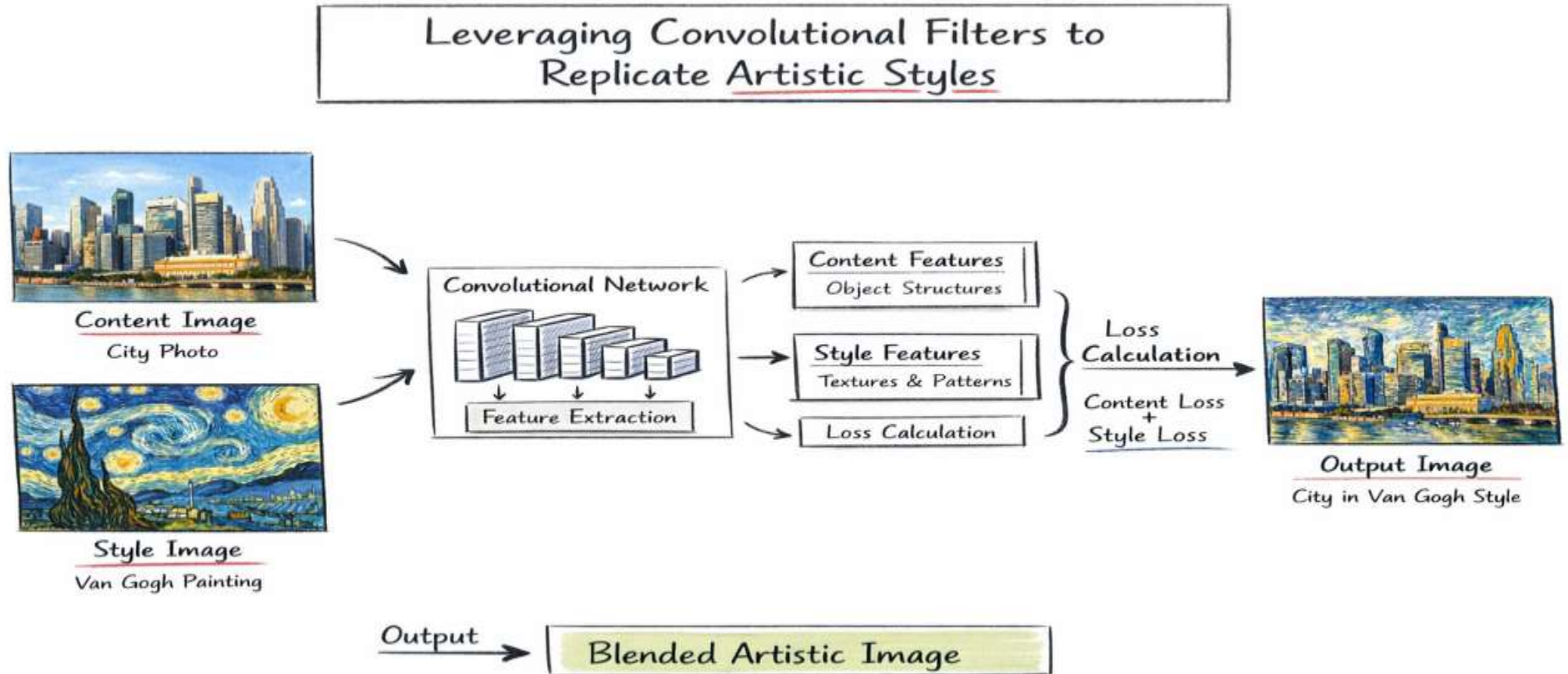
- Not always easy to interpret
  - May produce noisy or unclear visualizations
  - Computationally expensive
- 

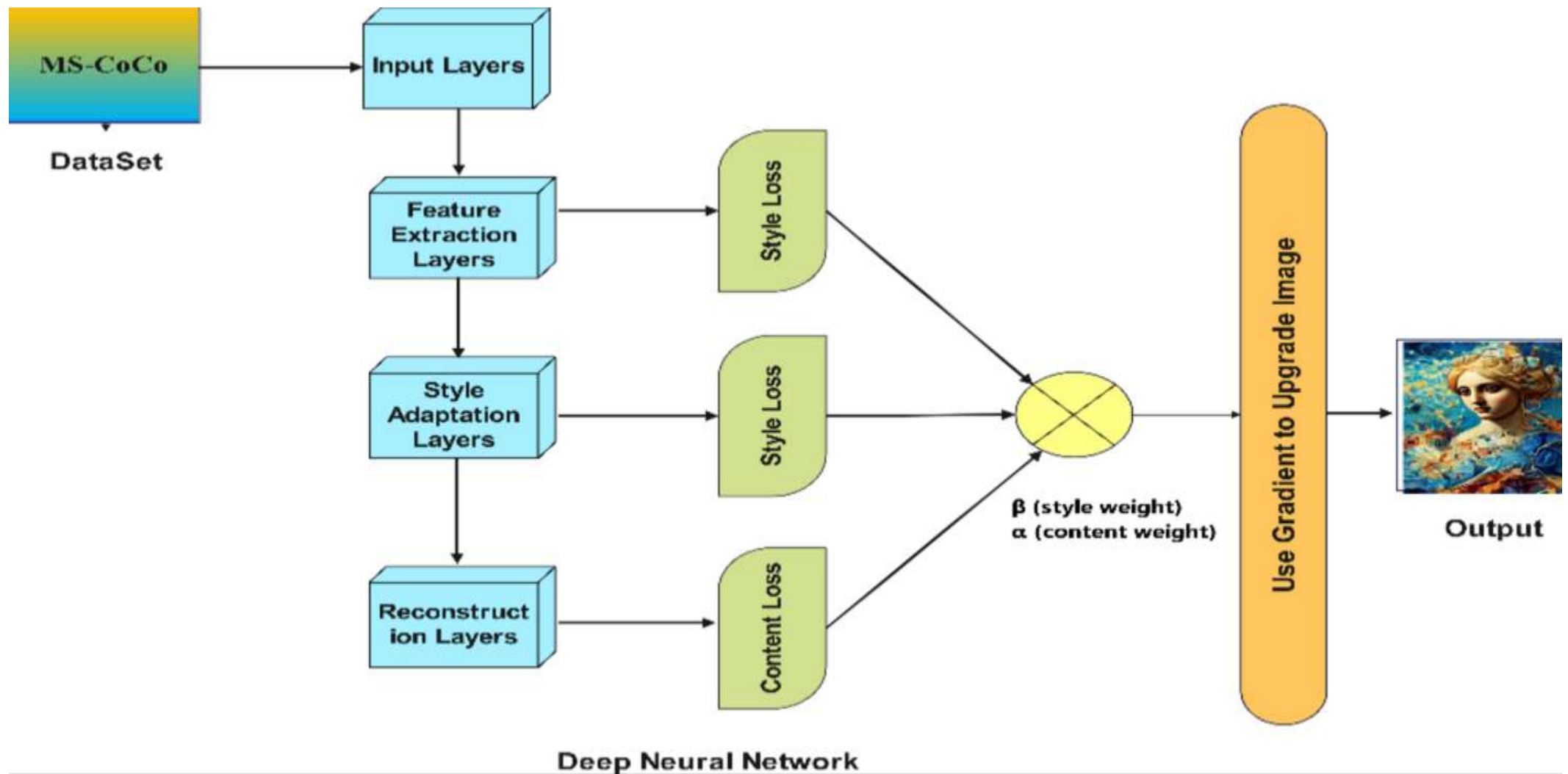
## 8. Applications

- Medical imaging (tumor detection explanation)
- Autonomous vehicles (object focus area)
- Face recognition systems
- AI research and debugging



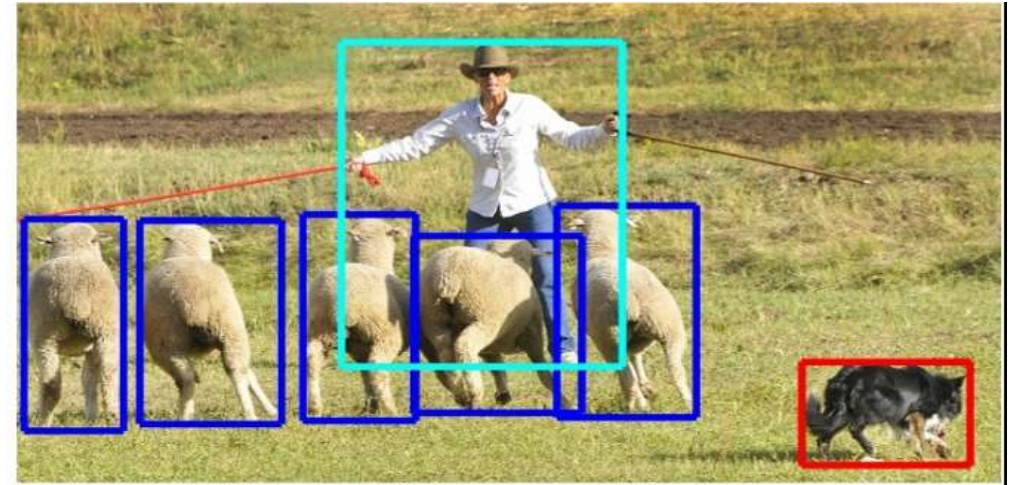
# 9. Leveraging Convolutional Filters to Replicate Artistic Styles







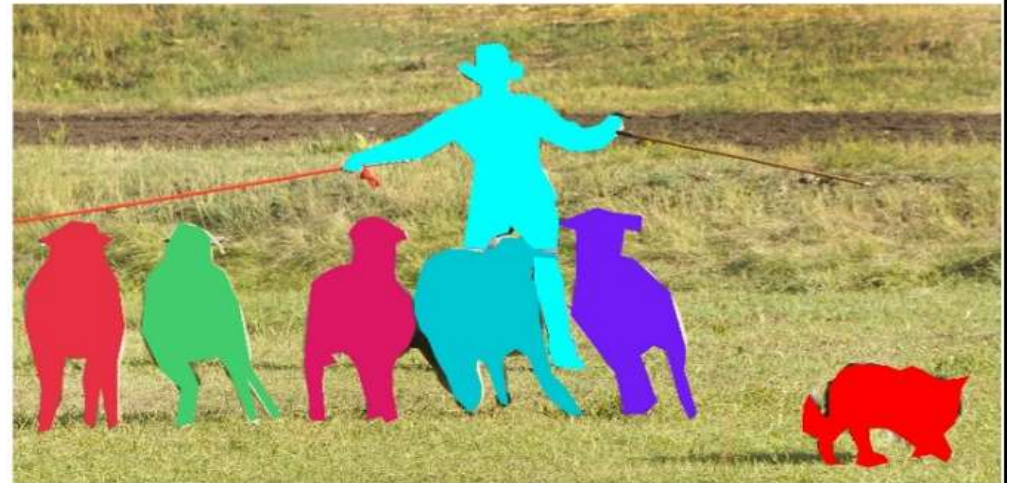
(a) Image classification



(b) Object localization



(c) Semantic segmentation



(d) This work



# 1. Introduction

**Leveraging convolutional filters to replicate artistic styles** refers to using **Convolutional Neural Networks (CNNs)** to transform an image so that it **preserves its content but adopts the artistic style of another image**.

This technique is popularly known as **Neural Style Transfer (NST)** in Deep Learning.

## 2. Basic Idea

We combine:

**Content Image** → structure (objects, shapes)

**Style Image** → texture, colors, brush strokes

Output = Stylized Image.

### 3. Role of Convolutional Filters

CNN filters extract different types of information:

#### **Content Representation**

Extracted from **deeper layers**

Captures:

- Shapes
- Object structure

#### **Style Representation**

- Extracted from **multiple layers**
- Captures:
  - Textures
  - Patterns
  - Colors

Style is represented using **Gram Matrix** (correlation between feature maps)

#### 4. Working of Neural Style Transfer(NST)

##### Step 1: Input Images

Content image (e.g., a photo)

Style image (e.g., painting)

##### Step 2: Pass through CNN

Use a pretrained network (like VGG)

##### Step 3: Extract Features

Content features from deeper layers

Style features from multiple layers

##### Step 4: Compute Loss

###### Content Loss

- Difference between content features

###### Style Loss

- Difference between Gram matrices

###### Total Loss

$$\text{Total Loss} = \text{Content Loss} + \text{Style Loss}$$

##### Step 5: Optimization

Start with random or content image .

Update pixels using gradient descent .

Minimize total loss .

## **5. Example**

Content image: Person photo .

Output:

Same person image.

With swirling brush strokes and colors of the painting.

## **6. Key Concepts**

### **1. Feature Maps**

Represent extracted patterns.

### **2. Gram Matrix**

Captures style (texture relationships).

### **3. Pretrained Networks**

Commonly used VGG-19.

## **7. Advantages**

1. Produces visually appealing results .
2. Demonstrates the power of CNN feature extraction.
3. No need for labeled data.

## **8. Limitations**

1. Computationally expensive.
2. May distort content if not balanced properly
3. Requires tuning of loss weights.

## **9. Applications**

1. Digital art creation.
2. Photo editing apps.
3. Film and animation.



# 10. Learning Convolutional Filters for Other Problem Domains

## 1. Definition

Convolutional filters are the core components of **Convolutional Neural Networks (CNNs)**. Although originally designed for image processing, these filters can be extended to **other domains to learn patterns from data automatically**. This capability makes CNNs highly powerful in Deep Learning.

## 2. Concept of Convolutional Filters

A **convolutional filter (kernel)** is a small matrix that moves across input data and performs:

- Element-wise multiplication
- Summation
- The result is a **feature map** that highlights important patterns such as edges, trends, or correlations.

### **3. Generalization of Convolution Across Domains**

- Convolution is not limited to images. It can be applied to different data structures:

#### **(1)1D Convolution**

Used for sequential data

Input: 1D signals

Applications:

Time-series (stock prices, weather data)

Audio processing

Text analysis

Learns temporal patterns and trends.

## **(2) 2D Convolution**

Used for image data

Input: 2D matrix (height  $\times$  width)

Applications:

Image classification

Object detection

## **(3) 3D Convolution**

Used for volumetric or temporal data

Applications:

Video processing (frame sequences)

Medical imaging (MRI, CT scans)

## **4. Applications in Various Domains**

- Natural Language Processing (NLP).
- Speech Recognition.
- Time-Series Analysis.
- Healthcare and Medical Field.
- Video and Multimedia Processing.

## **5. Working Mechanism**

1. Input data (text, audio, image, etc.)
2. Apply convolution filters
3. Generate feature maps
4. Apply activation function (ReLU)
5. Pooling reduces dimensions
6. Fully connected layer for classification

## **6. Advantages**

1. Automatic feature extraction
2. Reduces manual effort
3. Works across multiple domains
4. Captures local dependencies effectively
5. High accuracy in complex tasks

## **7. Challenges**

1. Requires large datasets
2. Computationally expensive
3. Complex model tuning
4. Difficult to interpret

# 11. Training algorithms.

## 1. Introduction

**Training algorithms** are methods used to **train neural networks** by adjusting weights and biases to minimize error. They play a crucial role in learning patterns from data in Deep Learning.

## 2. Objective of Training

The main goal is to:

- I. Minimize the **loss (error) function** .
- II. Improve model accuracy.
- III. Learn optimal parameters.

### **3. Steps in Training a Neural Network**

#### **Step 1: Forward Propagation**

Input data is passed through the network

Output is generated

#### **Step 2: Loss Calculation**

Compare predicted output with actual output

Compute error using loss functions

Examples:

Mean Squared Error (MSE)

Cross-Entropy Loss

#### **Step 3: Backpropagation**

Error is propagated backward

Gradients are calculated using chain rule

#### **Step 4: Weight Update**

Weights are updated using optimization algorithms



- **4. Gradient Descent Algorithm**
- **Definition**
- An optimization algorithm used to **minimize the loss function**.
- **Formula**
- $$W = W - \eta \frac{dL}{dW}$$
- Where:
- $W$  = weights
- $\eta$  = learning rate
- $L$  = loss function

## **5. Types of Gradient Descent:- 3 Types**

### **1. Batch Gradient Descent**

- Uses entire dataset
- Accurate but slow

### **2. Stochastic Gradient Descent (SGD)**

- Uses one data point at a time
- Faster but noisy

### **3. Mini-Batch Gradient Descent**

- Uses small batches
- Balance between speed and accuracy

## **6. Techniques to Improve Training**

- Regularization (Dropout, L2)
- Batch Normalization
- Learning rate scheduling
- Data augmentation

## **7. Applications**

- Training algorithms are used in:
- Image classification
- Speech recognition
- Natural language processing
- Medical diagnosis